



Objective & Lecture Mapping: In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment [cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

Informed search algorithms require a heuristic function, $h(n)$, which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called `Week_04` in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class [cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula $h(n) = |x_1 - x_2| + |y_1 - y_2|$ and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position `(0, 0)` and goal `(3, 4)` to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

A* evaluates nodes by combining the path cost so far, $g(n)$, and the estimated cost to the goal, $h(n)$. The evaluation function is $f(n) = g(n) + h(n)$.

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses $g(n)$, your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, $g(n) = 0$. Calculate $h(n)$ using your chosen heuristic method, and set $f(n) = g(n) + h(n)$.
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate $g_{\text{new}} = g_{\text{current}} + 1$, h_{new} using your heuristic, and $f_{\text{new}} = g_{\text{new}} + h_{\text{new}}$. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.

2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.
5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

The key difference is how they calculate the priority of a node. **Uniform-Cost Search (UCS)** prioritizes nodes using only the path cost so far, $g(n)$. Therefore, it always expands the node with the lowest cost from the starting state. **A* Search**, on the other hand, uses both the cost so far and a heuristic estimate of the remaining cost to the goal: $f(n) = g(n) + h(n)$. Therefore, A* considers both the distance already travelled and how promising the node appears to be toward the goal. According to Lecture 05, this allows A* to focus the search toward the goal while maintaining optimality when an appropriate heuristic is used.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

Manhattan Distance is admissible for this 4-way movement grid because the agent can only move **Up, Down, Left, or Right**. The Manhattan distance,

$$h(n) = |x_1 - x_2| + |y_1 - y_2|,$$

represents the minimum number of horizontal and vertical moves required to reach the goal if there were no walls. Walls can only force the agent to take additional moves; they cannot make the actual path shorter than the Manhattan distance. Therefore, Manhattan Distance never overestimates the true cost to the goal, satisfying the definition of an **admissible heuristic** from Lecture 05. If the heuristic were not admissible and overestimated the true cost, A* could prioritize nodes incorrectly and could return a suboptimal path, so its optimality would no longer be guaranteed.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

If the grid allowed **8-way movement**, including diagonal moves, Manhattan Distance would no longer necessarily be admissible if a diagonal move has a cost of 1. For example, moving diagonally from $(0,0)$ to $(3,3)$ could take only 3 diagonal moves, while Manhattan Distance gives $3 + 3 = 6$. In this case, Manhattan Distance overestimates the actual cost and is therefore not admissible.

For an 8-way grid where diagonal and straight moves have equal cost, a suitable heuristic would be **Chebyshev Distance**:

$$h(n) = \max(|x_1 - x_2|, |y_1 - y_2|).$$

If diagonal movement has a cost of $\sqrt{2}$, **Euclidean Distance** would be an appropriate admissible heuristic because it gives the straight-line distance and does not overestimate the shortest possible movement cost.

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food

efficiently.

When there are multiple food items, using only the distance to the closest food is weak because it ignores the remaining food items and the cost of travelling between them. A stronger heuristic would estimate the minimum total distance required to visit **all remaining food items**. One approach is to calculate a **Minimum Spanning Tree (MST)** connecting the agent's current position and all remaining food positions, using Manhattan distance as the edge cost. The total MST cost gives a lower-bound estimate of the distance required to connect all the food items. Since Manhattan distance never overestimates the actual 4-way movement cost, this provides a stronger admissible heuristic than simply considering the closest food. It gives A* information about the overall remaining task rather than only the next food item.

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation



FACULTY OF COMPUTING